

LABORATORY REPORT

BCA152 – Real-Time Operating Systems

Laboratory Activity No. 1:

Real-Time Multisensor Room Monitoring System

Submitted by:

Aiza Kate D. Buale

B186

Submitted to:

Paul Rodolf P. Castor

Date:

09/24/2026

PROBLEM AND REQUIREMENTS

2.1 Problem Statement

This lab's objective is to design and construct a real-time room monitoring system from the scratch. The system needs to be simulated in Wokwi and run on an ESP32 with ESP-IDF and FreeRTOS. The system must keep an eye on the room's conditions, show the readings, let the user switch between measurements, and sound an alert when the temperature goes out of range.

Additionally, the system needs to be able to determine whether the room is in use or not. The system should enter an inactive state if motion is not detected for a predetermined amount of time. The system ought to reactivate when motion resumes.

2.2 Required Behavior

The system must meet the following functional requirements:

- FR-01: Measure temperature periodically.
- FR-02: Measure humidity periodically.
- FR-03: Monitor ambient light level.
- FR-04: Detect motion using a PIR sensor.
- FR-05: Display one measurement at a time on an OLED screen.

- FR-06: Use a rotary encoder to switch between Temperature, Humidity, Light, and Motion.
- FR-07: Activate a buzzer when temperature goes below 18 °C or above 30 °C.
- FR-08: Support ACTIVE and INACTIVE states.
- FR-09: Enter INACTIVE state after no motion for 15 seconds.
- FR-10: Return to ACTIVE state when motion is detected again.

2.3 Wokwi Adaptation

Wokwi was used to simulate the entire system because a physical ESP32 was not available. The DHT22 sensor, LDR, PIR sensor, rotary encoder, SSD1306 OLED, and buzzer are all simulated by Wokwi. The code can be tested before running on actual hardware because the simulation behaves exactly like a real circuit.

3. SYSTEM ARCHITECTURE AND DESIGN

3.1 Hardware Architecture

The system is built around an ESP32 DevKit. It has following components:

- DHT22 sensor for temperature and humidity
- LDR (photoresistor) for ambient light
- PIR sensor for motion detection
- Rotary encoder for user navigation
- SSD1306 OLED for display
- Buzzer for the alarm

GPIO 4 is connected to the DHT22. GPIO 34 (ADC) is connected to the LDR. GPIO 13 is connected to the PIR sensor. GPIO 14 and GPIO 27 contain the encoder's CLK and DT pins. I2C is used by the OLED on GPIOs 21 (SDA) and 22 (SCL). The LEDC PWM peripheral drives the buzzer, which is located on GPIO 18.

3.2 Software Architecture

The firmware is divided into several modules:

- main.c – initializes the RTOS objects and creates all tasks
- tasks.c – contains the body of all six FreeRTOS tasks
- dht22.c, ldr.c – sensor drivers
- oled.c – SSD1306 display driver

- alarm.c – alarm logic and buzzer control
- input.c – encoder logic and display mode transitions
- system_state.c – pure function for the ACTIVE/INACTIVE state machine

This separation keeps hardware drivers apart from decision logic. Functions like evaluateTemperature(), nextDisplayMode(), and evaluateSystemState() do not touch any hardware and can be tested on the PC.

3.3 Subsystem Decomposition

The system has four (4) subsystems:

- Sensor subsystem: reads DHT22, LDR, and PIR.
- Display subsystem: owns the OLED and shows the current measurement.
- Input subsystem: reads the rotary encoder and updates the display mode.
- Alarm subsystem: evaluates temperature and controls the buzzer.

3.4 State Machine

The system has two (2) states:

- ACTIVE: OLED is on, sensor processing runs, encoder navigation works, and the alarm is active.
- INACTIVE: Motion detection continues to function, but the OLED is blank, sensor display functions are diminished, and the buzzer is turned off.

Transitions:

When motion is detected, INACTIVE becomes ACTIVE.

After 15 seconds of not moving, ACTIVE becomes INACTIVE.

4. FREERTOS ARCHITECTURE

4.1 Tasks

The system uses six FreeRTOS tasks. Each task has a specific job, a priority, and a way to communicate with the other tasks.

Task	Responsibility	Trigger / Period	Priority	IPC	Blocked Condition
MotionTask	Reads PIR sensor	100 ms	3	Event group	vTaskDelayUntil
InputTask	Reads rotary encoder	10 ms	3	Mutex (xDisplayModeMutex)	vTaskDelayUntil
SensorReadTask	Reads DHT22 and LDR	2000 ms	2	Queues (xSensorQueue, xAlarmQueue)	vTaskDelayUntil
AlarmTask	Controls buzzer	Queue-driven	2	Queue (xAlarmQueue)	xQueueReceive
StateTask	Manages ACTIVE / INACTIVE	100 ms	2	Event group	vTaskDelayUntil
DisplayTask	Controls OLED	Queue-driven	1	Queue (xSensorQueue)	xQueueReceive

4.2 Priority Justification

MotionTask and InputTask have the highest priority (3) because they need the fastest response. The system will feel slow and the inactivity timer may expire before motion is detected if motion detection or encoder input are delayed.

Priority 2 is assigned to SensorReadTask, AlarmTask, and StateTask. They perform bounded latency work on a periodic basis. They have space because of the 2-second sensor period, but alarms must still respond before the subsequent sensor cycle.

The least important task is DisplayTask (1). The task only executes when fresh data comes in from the queue, and a slight delay before the OLED refreshes has no effect on the system.

4.3 Task States

At any moment, a task can be in one of these states:

- Running: currently using the CPU.
- Ready: able to run but waiting for the CPU.
- Blocked: waiting for an event, queue, or delay. Does not consume CPU.
- Suspended: explicitly paused (not used in this project).
- Deleted: removed (not used in this project).

For example, SensorReadTask is Running while it reads the DHT22, then Blocked while it waits inside vTaskDelayUntil for the next 2-second cycle. When the delay ends, it becomes Ready and waits for the CPU.

4.4 Inter-Task Communication

The system uses three (3) types of FreeRTOS synchronization objects:

- Queues (xSensorQueue, xAlarmQueue): pass sensor_data_t structs from SensorReadTask to DisplayTask and AlarmTask. Queue length is 10 items each.
- Mutexes (xSerialMutex, xDisplayModeMutex): protect shared data. xSerialMutex protects printf() calls. xDisplayModeMutex protects the shared currentDisplayMode variable.
- Event group (xSystemEventGroup): signals state changes across tasks. Bits used:
 - BIT_MOTION_DETECTED (bit 0) – set by MotionTask, read by StateTask and SensorReadTask.
 - BIT_ALERT_TRIGGERED (bit 1) – set by AlarmTask when temperature is out of range.
 - BIT_SYSTEM_ACTIVE (bit 2) – set or cleared by StateTask, read by DisplayTask, InputTask, and AlarmTask.

4.5 Periodic Execution with vTaskDelayUntil()

MotionTask and InputTask have the highest priority (3) because they need the fastest response. The subsequent wake time is anchored to a predetermined tick count by this function. This implies that even if a task takes longer than anticipated, the period does not change. vTaskDelay() would gradually drift the delay by adding the task's execution time.

5. IMPLEMENTATION

5.1 Sensor Reading

SensorReadTask uses vTaskDelayUntil() to read the DHT22 and LDR every two seconds. A unique bit-banging driver on GPIO 4 is used to read the DHT22. The ESP32 ADC on GPIO 34 reads the LDR and converts it to a 0-100% value. Following reading, the task sends the data to xSensorQueue and xAlarmQueue after packaging it into a sensor_data_t struct.

5.2 OLED Display

DisplayTask owns the OLED. No other task writes to the display. The task waits on xSensorQueue and, when data arrives, checks the BIT_SYSTEM_ACTIVE event bit. If the system is INACTIVE, the task skips the update. If the system is ACTIVE, the task reads the current display mode (protected by xDisplayModeMutex) and draws the appropriate screen. The OLED driver implements a small 5x7 font and a framebuffer that is sent to the SSD1306 over I2C.

5.3 Rotary Encoder

Every ten milliseconds, InputTask reads the encoder's CLK and DT pins. The task determines the direction by reading the DT pin when the CLK pin changes from high to low. The next mode is reached in a clockwise direction; the previous mode is reached in an anticlockwise direction. The mode is kept in currentDisplayMode, which xDisplayModeMutex protects.

5.4 Alarm Logic

In alarm.c, the alarm logic is divided into a pure function called evaluateTemperature(). Based on the 18 °C and 30 °C limits, this function returns ALARM_NORMAL, ALARM_LOW_TEMP, or ALARM_HIGH_TEMP. AlarmTask calls evaluateTemperature(), waits on xAlarmQueue, and activates or deactivates the buzzer. The LEDC PWM peripheral powers the buzzer at a frequency of 2 kHz. The BIT_ALERT_TRIGGERED event bit is also set or cleared by AlarmTask.

5.5 System State

StateTask runs every 100 ms. It reads BIT_MOTION_DETECTED and keeps a counter of how long the system has been without motion. The counter is passed to evaluateSystemState(), which returns STATE_ACTIVE or STATE_INACTIVE. When the state changes, StateTask updates the BIT_SYSTEM_ACTIVE event bit and prints a message through xSerialMutex. The inactivity timeout is set to 15 seconds for testing.

5.6 Shared Resource Protection

Two (2) mutexes protect shared resources:

- xSerialMutex protects all printf() calls. Every task takes this mutex before printing.
- xDisplayModeMutex protects currentDisplayMode. InputTask writes to it; DisplayTask reads it.

Without these mutexes, output could be interleaved and currentDisplayMode could be read while being written.

6. VERIFICATION AND TESTING

6.1 Unit Testing

Unit tests were written for the hardware-independent logic. They run on the PC using the native PlatformIO environment, so no ESP32 is needed.

Command:

```
pio test -e native
```

Result: 13 out of 13 tests passed.

Coverage:

Temperature alarm logic (5 tests):

- Below lower threshold ($< 18\text{ }^{\circ}\text{C}$)
- Exactly at lower threshold ($= 18\text{ }^{\circ}\text{C}$)
- Normal value (between $18\text{ }^{\circ}\text{C}$ and $30\text{ }^{\circ}\text{C}$)
- Exactly at upper threshold ($= 30\text{ }^{\circ}\text{C}$)
- Above upper threshold ($> 30\text{ }^{\circ}\text{C}$)

Display navigation (4 tests):

- Forward transition
- Forward wraparound
- Reverse transition
- Reverse wraparound

System state (4 tests):

- ACTIVE without timeout
- ACTIVE with timeout (switches to INACTIVE)
- INACTIVE without motion (stays INACTIVE)
- INACTIVE with motion (switches to ACTIVE)

6.2 Functional Verification in Wokwi

Each functional requirement was tested in the Wokwi simulation. The table below summarizes the results.

Test ID	Stimulus	Expected	Actual	Result
FT-01	Change temperature	Displayed temperature updates	OLED showed new temperature	PASS
FT-02	Change humidity	Displayed humidity updates	OLED showed new humidity	PASS
FT-03	Change light input	Light value changes	OLED showed new light value	PASS
FT-04	Rotate encoder clockwise	Next page selected	Moved to next mode	PASS
FT-05	Rotate encoder counterclockwise	Previous page selected	Moved to previous mode	PASS
FT-06	Set temperature above 30 °C	Alarm activates	Buzzer turned on	PASS
FT-07	Return temperature to normal	Alarm stops	Buzzer turned off	PASS
FT-08	Trigger PIR	System is ACTIVE	System stayed ACTIVE	PASS
FT-09	Allow inactivity timeout	System becomes INACTIVE	OLED went blank	PASS
FT-10	Trigger PIR while INACTIVE	System returns to ACTIVE	OLED came back	PASS

All 10 functional tests passed.

Screenshots of all functional tests are stored in the docs/ folder of the repository. The list below shows the file names for each test.

- FT-01: docs/ft01-temp-before.png, docs/ft01-temp-after.png

- FT-02: docs/ft02-humidity-before.png, docs/ft02-humidity-after.png
- FT-03: docs/ft03-light-low.png, docs/ft03-light-high.png
- FT-04: docs/ft04-encoder-navigation-verified.png
- FT-05: docs/ft05-encoder-ccw.png
- FT-06: docs/ft06-alarm-normal.png
- FT-07: docs/ft07-alarm-high.png
- FT-08: docs/ft08-motion-active.png
- FT-09: docs/ft09-inactive-timeout.png
- FT-10: docs/ft10-motion-reactivation.png

6.3 Deliberate FreeRTOS Fault Experiments

Fault Experiment 1 – Remove Blocking

The blocking delay was temporarily removed from MotionTask. The task then ran in a tight loop and starved other tasks. The serial output stopped and the system became unresponsive. This shows why every continuously running task must block or yield. The delay was restored afterward.

Fault Experiment 2 – Change Priority

The DisplayTask priority was temporarily raised to 5. The OLED started to update too often and the sensor task missed some cycles. This shows that giving a non-urgent task a high priority can starve more important tasks. The priority was restored to 1 afterward.

Fault Experiment 3 – Remove Mutex

The xSerialMutex was temporarily removed from SensorReadTask. The serial output became interleaved: lines from different tasks mixed together. This shows why shared printf() calls must be protected. The mutex was restored afterward.

7. STATIC CODE ANALYSIS

Static code analysis was performed using cppcheck through PlatformIO.

Command:

```
pio check -e esp32dev
```

Result: PASSED – 0 HIGH, 0 MEDIUM, 20 LOW.

Findings and resolutions:

Finding	File / Line	Cause	Resolution
unusedFunction (20 findings)	alarm.c, dht22.c, input.c, ldr.c, main.c, oled.c, system_state.c, tasks.c	cppcheck analyzes each .c file separately and cannot see cross-file calls. Each function is used in another file.	Accepted as a false positive. Documented.
constVariable	src/oled.c:17	The buffer buf was not modified after initialization.	Declared as const uint8_t buf[2].
variableScope	src/tasks.c:260	The variable newMode was declared in the outer block but only used inside an if block.	Moved the declaration inside the if block.

The two real findings (constVariable and variableScope) were fixed. The remaining unusedFunction warnings are a known limitation of cppcheck when working with multi-file C projects.

8. ENGINEERING DISCUSSION

8.1 Trade-offs

Several engineering trade-offs were made in this project.

First, a decision was made to use one task per responsibility instead of grouping multiple jobs into one task. This makes the code easier to test and reason about, but it uses more RAM for task stacks. Since the ESP32 has 320 KB of RAM and each task stack is 3 KB, this trade-off was acceptable.

Second, the rotary encoder is polled every 10 ms instead of using GPIO interrupts. Polling is simpler to implement and works well in Wokwi, but on real hardware it uses more CPU time. An interrupt-based approach would be more power-efficient.

Third, queues were used for sensor data instead of global variables. Queues have a small overhead, but they provide automatic synchronization and blocking behavior, which makes the code safer.

8.2 Limitations

The system was tested only in Wokwi and not on real ESP32 hardware. The encoder is polled, not interrupt-driven. The LDR reading is relative (0-100%) and not calibrated in lux. The OLED shows only one measurement at a time. The cppcheck tool reports 20 unusedFunction warnings that are false positives.

8.3 Debugging and Problems Encountered

Several problems were encountered during development.

The first problem was the DHT22 sensor. The sensor requires precise timing for its one-wire protocol, and the first attempts returned NaN values. The issue was solved by adding proper microsecond delays in the bit-banging code.

The second problem was the OLED initialization. The SSD1306 needs a specific sequence of commands to turn on, and getting the I2C address wrong caused the display to stay blank. After checking the datasheet, the correct address (0x3C) was used.

The third problem was the encoder debouncing. The mechanical encoder generated multiple pulses for a single turn. This was solved by only reacting to the falling edge of the CLK pin.

The fourth problem was static analysis. The cppcheck tool reported 20 unusedFunction warnings. After investigation, it was found that cppcheck analyzes each .c file separately and cannot see cross-file calls. These are false positives and were documented.

8.4 Alternative Approaches

An alternative to the queue-based design would be to use global variables protected by a single mutex. This would use less memory but would require more careful coding to avoid deadlocks. The queue approach was chosen because it is safer and easier to reason about.

Another alternative would be to use a single task that does everything in a loop. This would be simpler but would not meet the laboratory requirements and would not demonstrate FreeRTOS concepts.

8.5 What Would Differ on Real Hardware

On real hardware, several things would change. The encoder would use GPIO interrupts instead of polling. The DHT22 timing might need adjustment because real sensors have tighter timing requirements. Power consumption could be reduced by using ESP32 light sleep when the system is INACTIVE. The LDR would be calibrated in lux using a known reference. A hardware watchdog would be added to detect task failures.

9. CONCLUSION

This laboratory successfully implemented a real-time multisensor room monitoring system on the ESP32 using ESP-IDF and FreeRTOS, and simulated it in Wokwi.

The system meets all ten functional requirements: it measures temperature, humidity, light, and motion; displays one measurement at a time on an OLED; lets the user navigate with a rotary encoder; sounds a buzzer when the temperature goes out of range; and manages ACTIVE and INACTIVE states based on motion.

From a FreeRTOS standpoint, the project demonstrates multiple tasks with explicit priorities, queue-based communication, mutex-protected shared resources, an event group for cross-task signaling, periodic execution with `vTaskDelayUntil()`, and a state machine driven by a pure decision function.

The unit tests (13/13 passing) verified the decision logic without any hardware. The functional tests in Wokwi verified the full system behavior. The fault experiments showed what happens when blocking, priorities, and mutexes are misused.

The main lesson learned is that FreeRTOS is not just about running multiple tasks. It is about deciding which task should run first, which task can wait, how tasks should share data, and how to prevent them from interfering with each other. These decisions have to be justified based on timing and safety, not just convenience.

For future improvement, the encoder could be interrupt-driven, the LDR could be calibrated in lux, the system could support Wi-Fi monitoring, and the temperature limits could be stored in NVS so they can be changed at runtime.

Requirements Traceability Matrix

Requirement	Implementation	Verification
FR-01	SensorTask	FT-01
FR-05	DisplayTask	FT-01–FT-03

FR-06	InputTask	Unit navigation tests + FT-04/FT-05
FR-07	AlarmTask	Unit alarm tests + FT-06/FT-07
FR-09	MotionTask / StateTask	Unit state tests + FT-09